05. STARK 증명 생성 (STARK Proof Generation)

5.1 STARK 개요

5.1.1 STARK란?

STARK (Scalable Transparent ARgument of Knowledge)는 다음 특징을 가진 영지식 증명 시스템입니다:

핵심 특징:

- **Scalable (확장 가능)**: 큰 계산도 효율적으로 증명
- **Transparent (투명)**: Trusted Setup 불필요
- **Post-Quantum (양자 내성)**: 양자 컴퓨터에도 안전
- **Succinct (간결)**: 증명 크기가 작음

5.1.2 STARK의 구성 요소

```
1. Execution Trace (실행 추적)

↓

2. Polynomial Commitment (다항식 커밋먼트)

↓

3. FRI Protocol (저차 테스트)

↓
```

4. Proof (증명)

5.2 증명 생성 함수 분석

5.2.1 generate_proof 함수 구조

```
impl VMAir {
pub fn generate_proof(&self, vm: VM) {
// 1. 타입 정의
type Val = Mersenne31;
type Challenge = BinomialExtensionField<Val, 3>;
// 2. 암호학적 구성 요소 설정
// 3. FRI 설정
// 4. PCS 설정
// 5. STARK 설정
// 6. 추적 생성
// 7. 증명 생성
// 8. 증명 검증
}
}
```

5.3 타입 정의

5.3.1 기본 필드

```
type Val = Mersenne31;
```

Mersenne31이란?

- 메르센 소수 기반 필드: $p = 2^{31} - 1$
- 32비트 정수로 효율적 연산
- 빠른 모듈로 연산

왜 Mersenne31을 사용하는가?

- CPU 친화적 (32비트 연산)
- 빠른 산술 연산
- 충분한 보안 수준

5.3.2 확장 필드

```
type Challenge = BinomialExtensionField<Val, 3>;
```

BinomialExtensionField<Val, 3>:

- Mersenne31의 3차 확장 필드
- 원소: $a_0 + a_1 \cdot \alpha + a_2 \cdot \alpha^2$ ($a_0, a_1, a_2 \in$ Mersenne31)
- 더 큰 보안 수준 제공

왜 확장 필드가 필요한가?

- 보안 강화: 더 큰 필드 크기
- FRI 프로토콜 요구사항
- 충돌 저항성 향상

5.4 암호학적 구성 요소

5.4.1 해시 함수 설정

```
type ByteHash = Keccak256Hash;

type FieldHash = SerializingHasher32<ByteHash>;

let  byte_hash = ByteHash {};

let  field_hash = FieldHash::new(byte_hash);
```

Keccak256Hash:

- SHA-3 표준 해시 함수
- 256비트 출력
- 암호학적으로 안전

SerializingHasher32:

- 필드 원소를 바이트로 직렬화
- Keccak256으로 해싱
- 32비트 필드 원소에 최적화

5.4.2 압축 함수

```
type MyCompress = CompressionFunctionFromHasher<ByteHash, 2, 32>;

let  compress = MyCompress::new(byte_hash);
```

CompressionFunctionFromHasher:

- 두 개의 해시 값을 하나로 압축

- 머클 트리 구성에 사용
- 파라미터: `<해시함수, 입력개수, 출력크기>`

5.5 머클 트리 커밋먼트 스킴 (MMCS)

5.5.1 ValMmcs 설정

```
type ValMmcs = MerkleTreeMmcs<Val, u8, FieldHash, MyCompress, 32>;

let  val_mmcs = ValMmcs::new(field_hash, compress);
```

MerkleTreeMmcs:

- Merkle Tree 기반 다항식 커밋먼트
- 추적 데이터를 머클 트리로 커밋

구조:

```
Root (커밋먼트)

/ \

H1 H2

/ \ / \

L1 L2 L3 L4 (추적 데이터)
```

타입 파라미터:

- `Val` : 필드 타입 (Mersenne31)
- `u8` : 해시 출력 타입
- `FieldHash` : 필드 해싱 함수

- `MyCompress` : 압축 함수
- `32` : 해시 출력 크기

5.5.2 ChallengeMmcs 설정

```
type ChallengeMmcs = ExtensionMmcs<Val, Challenge, ValMmcs>;

let  challenge_mmcs = ChallengeMmcs::new(val_mmcs.clone());
```

ExtensionMmcs:

- 확장 필드용 MMCS
- FRI 프로토콜에서 사용
- 기본 MMCS를 확장 필드로 확장

5.6 Challenger (도전자)

5.6.1 Challenger 설정

```
type Challenger = SerializingChallenger32<Val, HashChallenger<u8, ByteHash, 32>>

let  mut  challenger = Challenger::from_hasher(vec![], byte_hash);
```

Challenger의 역할:

- Fiat-Shamir 변환 구현
- 상호작용을 비상호작용으로 변환
- 무작위 도전 값 생성

Fiat-Shamir 변환:

```
상호작용 프로토콜:

Prover → Commitment → Verifier

Verifier → Challenge → Prover

Prover → Response → Verifier



↓ Fiat-Shamir 변환



비상호작용 프로토콜:

Prover → Commitment

Challenge = Hash(Commitment)

Prover → Response
```

5.6.2 도전 값 생성 과정

```
challenger.observe(commitment); // 커밋먼트 관찰

let  challenge = challenger.sample(); // 도전 값 샘플링
```

과정:

1. 증명자가 커밋먼트 생성
2. Challenger가 커밋먼트를 해싱
3. 해시 값으로부터 도전 값 생성
4. 도전 값은 예측 불가능하고 결정론적

5.7 FRI 설정

5.7.1 FRI 개념

FRI (Fast Reed-Solomon Interactive Oracle Proofs)는 다항식이 낮은 차수임을 증명하는 프로토콜입니다.

핵심 아이디어:

- 다항식의 차수를 반복적으로 줄임
- 각 단계에서 커밋먼트 생성
- 최종적으로 상수 다항식에 도달

5.7.2 FRI 설정 코드

```
let  fri_config = FriConfig {
log_blowup: 1,
num_queries: 40,
proof_of_work_bits: 8,
mmcs: challenge_mmcs,
};
```

파라미터 설명:

log_blowup: 1

```
blowup_factor = 2^log_blowup = 2^1 = 2
```

- 다항식 평가 도메인의 확장 비율

- 추적 크기의 2배 도메인에서 평가
- 보안과 효율성의 균형

왜 확장하는가?

- 낮은 차수 테스트의 정확성 향상
- 다항식 보간의 유일성 보장

num_queries: 40

- FRI 프로토콜에서 수행할 쿼리 수
- 더 많은 쿼리 = 더 높은 보안
- 40개 쿼리 ≈ 80비트 보안

쿼리 과정:

```
각 쿼리마다:
1. 무작위 위치 선택
2. 해당 위치의 값과 증명 요청
3. 머클 경로 검증
```

proof_of_work_bits: 8

- 작업 증명 난이도
- 2^8 = 256번의 해시 시도 필요
- 무차별 대입 공격 방지

작업 증명:

```
nonce = 0
```

```
while hash(proof || nonce)의 앞 8비트 != 0:
nonce += 1
```

mmcs: challenge_mmcs

- 확장 필드용 MMCS 사용
- FRI 커밋먼트에 사용

5.8 PCS (Polynomial Commitment Scheme)

5.8.1 CirclePcs 설정

```
type Pcs = CirclePcs<Val, ValMmcs, ChallengeMmcs>;
let pcs = Pcs {
mmcs: val_mmcs,
fri_config,
_phantom: PhantomData,
};
```

CirclePcs:

- Circle STARK의 다항식 커밋먼트 스킴
- FRI 기반 커밋먼트
- 효율적인 증명 생성

구성 요소:

- `mmcs` : 기본 필드용 MMCS

- `fri_config` : FRI 설정
- `_phantom` : 타입 파라미터 유지용

5.8.2 PCS의 역할

```
1. Commit Phase (커밋 단계):
Polynomial → Evaluation → Merkle Tree → Root (Commitment)

2. Open Phase (오픈 단계):
Query → Evaluation + Merkle Proof → Verification
```

5.9 STARK 설정

5.9.1 StarkConfig 생성

```rust
type MyConfig = StarkConfig<Pcs, Challenge, Challenger>;
let  config = MyConfig::new(pcs);
```

StarkConfig:

- STARK 증명 시스템의 전체 설정
- PCS, 도전자, 필드 타입 통합

타입 파라미터:

- `Pcs` : 다항식 커밋먼트 스킴
- `Challenge` : 도전 값 타입 (확장 필드)

- `Challenger` : 도전자 타입

5.10 증명 생성

5.10.1 prove 함수 호출

```
let trace = get_trace(vm);

let proof = prove(&config, self, &mut challenger, trace, &vec![]);
```

prove 함수 파라미터:

- `config` : STARK 설정
- `self` : VMAir (AIR 정의)
- `challenger` : 도전자
- `trace` : 실행 추적 (RowMajorMatrix)
- `&vec![]` : 공개 입력 (이 구현에서는 없음)

5.10.2 증명 생성 과정 (내부)

```
1. 추적 다항식화

Trace → Polynomials (각 컬럼 → 다항식)


2. 추적 커밋먼트

Polynomials → Evaluations → Merkle Tree → Commitment


3. AIR 제약 조건 평가
```

각 행에서 제약 조건 계산

Constraint Polynomial 생성

4. 제약 다항식 커밋먼트

Constraint Polynomial → Commitment

5. FRI 증명 생성

낮은 차수 증명

반복적 폴딩 및 커밋먼트

6. 쿼리 응답 생성

무작위 위치에서 값과 머클 경로 제공

7. 최종 증명 구성

모든 커밋먼트, 응답, 경로 결합

5.10.3 증명 구조

```
pub struct Proof<T> {
// 추적 커밋먼트
trace_commitments: Vec<Commitment>,
// 제약 다항식 커밋먼트
quotient_commitments: Vec<Commitment>,
```

```
// FRI 증명
fri_proof: FriProof,
// 쿼리 응답
query_openings: Vec<QueryOpening>,
// 작업 증명
pow_nonce: u64,
}
```

5.11 증명 검증

5.11.1 verify 함수 호출

```
verify(&config, self, &mut  challenger, &proof, &vec![]);
```

verify 함수 파라미터:

- `config` : STARK 설정 (증명 생성과 동일)
- `self` : VMAir (AIR 정의)
- `challenger` : 새로운 도전자 인스턴스
- `proof` : 생성된 증명
- `&vec![]` : 공개 입력

5.11.2 검증 과정

```
1. 도전 값 재생성
```

Challenger로 동일한 도전 값 생성

(Fiat-Shamir 변환의 결정론성)

2. 작업 증명 검증

pow_nonce가 유효한지 확인

3. 커밋먼트 검증

추적 및 제약 다항식 커밋먼트 확인

4. 쿼리 검증

각 쿼리 위치에서:

- 머클 경로 검증

- 제약 조건 만족 확인

5. FRI 검증

낮은 차수 증명 검증

각 폴딩 단계 확인

6. 최종 판정

모든 검증 통과 → Accept

하나라도 실패 → Reject

5.11.3 검증의 효율성

시간 복잡도:

- 증명 생성: O(n log n) (n = 추적 크기)
- 증명 검증: O(log n) (쿼리 수는 상수)

공간 복잡도:

- 증명 크기: O(log n)

5.12 전체 증명 흐름 다이어그램

```
┌─────────────────────────────────────────────────────────────┐
│ STARK 증명 생성 흐름 │
└─────────────────────────────────────────────────────────────┘

1. VM 실행
Program → VM.run() → Trace (Vec<VMState>)
2. 추적 변환
Trace → get_trace() → RowMajorMatrix<F>
3. 추적 다항식화
RowMajorMatrix → Interpolate → Polynomials
4. 추적 커밋먼트
Polynomials → Evaluate → Merkle Tree → Commitment₁
5. 도전 값 생성
Challenger.observe(Commitment₁ )
α = Challenger.sample()
```

```
6. 제약 다항식 생성

AIR.eval() → Constraint Polynomials

7. 제약 커밋먼트

Constraint Polynomials → Merkle Tree → Commitment₂

8. FRI 증명

Polynomials → FRI Protocol → FRI Proof

9. 쿼리 응답

Random Queries → Openings + Merkle Paths

10. 증명 구성

All Components → Final Proof

11. 검증

Proof + AIR → Verify → Accept/Reject
```

5.13 보안 분석

5.13.1 건전성 (Soundness)

정리: 잘못된 추적에 대해 증명을 생성할 확률은 무시할 수 있을 정도로 작다.

이유:

- FRI 프로토콜의 건전성
- 충분한 쿼리 수 (40개)
- 작업 증명 (2^8 난이도)

보안 수준:

```
Security ≈ 2^(-num_queries) × 2^(-pow_bits)

≈ 2^(-40) × 2^(-8)

≈ 2^(-48)
```

5.13.2 완전성 (Completeness)

정리: 올바른 추적에 대해 항상 증명을 생성할 수 있다.

이유:

- 올바른 추적은 모든 AIR 제약 조건 만족
- 제약 다항식이 올바르게 구성됨
- FRI 프로토콜이 성공적으로 완료

5.13.3 영지식성 (Zero-Knowledge)

주의: 이 구현은 영지식성을 제공하지 않습니다!

이유:

- 추적이 그대로 노출됨
- 무작위화 없음

영지식성 추가 방법:

- 추적에 무작위 값 추가
- 블라인딩 다항식 사용
- 추가 프로토콜 라운드

5.14 성능 최적화

5.14.1 병렬화

Plonky3는 다음을 병렬화합니다:

- 다항식 평가
- 머클 트리 구성
- FRI 폴딩

5.14.2 메모리 효율성

- 스트리밍 방식의 머클 트리 구성
- 불필요한 중간 데이터 제거
- 효율적인 필드 연산

5.15 실제 사용 예시

5.15.1 테스트 코드

```
#[test]
fn  test_end_to_end() {
let  program = vec![
Instructions::Push(Mersenne31::from_canonical_u32(10)),
Instructions::Push(Mersenne31::from_canonical_u32(20)),
Instructions::Add,
Instructions::Push(Mersenne31::from_canonical_u32(40)),
Instructions::Sub,
Instructions::Push(Mersenne31::from_canonical_u32(2)),
```

```
Instructions::Mul,
Instructions::Push(Mersenne31::from_canonical_u32(23)),
Instructions::Div,
];
let mut vm = VM::new(program);
vm.run().unwrap();
let vmair = VMAir {};
vmair.generate_proof(vm); // 증명 생성 및 검증
}
```

5.15.2 실행 결과

```
1. VM 실행 완료
2. 추적 생성 (9개 명령어 + 패딩)
3. 증명 생성 시작
4. FRI 프로토콜 실행
5. 쿼리 응답 생성
6. 증명 완성
7. 검증 시작
8. 모든 검증 통과
9. 성공!
```

5.16 요약

STARK 증명 생성은 복잡하지만 체계적인 과정입니다:

1. **설정**: 암호학적 구성 요소 초기화
2. **커밋먼트**: 추적을 머클 트리로 커밋
3. **제약 평가**: AIR 제약 조건 계산
4. **FRI**: 낮은 차수 증명 생성
5. **쿼리**: 무작위 위치 검증
6. **검증**: 모든 구성 요소 확인

이 과정을 통해 프로그램 실행의 정확성을 수학적으로 증명할 수 있습니다. 다음 문서에서는 전체 시스템을 통합하여 어떻게 동작하는지 살펴보겠습니다.